

HW 3

Sepehr Ilami

November 2025

Github Link: https://github.com/sepehrilami/phys7332_fa25/blob/main/assignments/assignment03.ipynb

Question 1

a)

```

1 import networkx as nx
2 import numpy as np
3 from collections import defaultdict
4 import scipy.sparse as sp
5 from networkx.algorithms import community
6 import time
7 import seaborn as sns
8 import pandas as pd
9 import os
10 import warnings
11 warnings.filterwarnings("ignore")
12 import random
13 import scipy.io
14 import graph_tool.all as gt
15 from sklearn.metrics import normalized_mutual_info_score
16 import matplotlib.pyplot as plt
17 from matplotlib.ticker import ScalarFormatter, LogLocator
18 import matplotlib as mpl
19
20 mpl.rcParams['font.family'] = 'sans-serif'
21 mpl.rcParams['font.sans-serif'] = ['Arial', 'DejaVu Sans']
22 mpl.rcParams['font.size'] = 11
23 mpl.rcParams['axes.linewidth'] = 1.2
24 mpl.rcParams['xtick.major.width'] = 1.2
25 mpl.rcParams['ytick.major.width'] = 1.2
26 mpl.rcParams['xtick.minor.width'] = 0.8
27 mpl.rcParams['ytick.minor.width'] = 0.8
28

```

```

1
2
3 def calculate_modularity(G):
4     """Calculate modularity using Louvain community detection"""
5     try:
6         communities = community.louvain_communities(G, seed=42)
7         modularity = community.modularity(G, communities)
8         return modularity
9     except:
10         return 0
11
12 def theoretical_curve(S, m, a=0.165):
13     """Theoretical prediction following Equation format"""
14     return (a + (1 - a)/m)*(1 - (2/np.sqrt(S)))
15
16 def generate_random_graph(n, m_param):
17     """Generate a random graph with n nodes and edge density based on m"""
18     # For random graphs, we interpret m as controlling the edge density
19     # Higher m means more edges per node on average
20     avg_degree = 2 * m_param # Average degree
21     num_edges = int(n * avg_degree / 2)
22
23     # Ensure we don't exceed maximum possible edges
24     max_edges = n * (n - 1) // 2
25     num_edges = min(num_edges, max_edges)
26
27     # Generate random graph
28     G = nx.gnm_random_graph(n, num_edges, seed=None)
29
30     # Ensure the graph is connected for meaningful modularity
31     if not nx.is_connected(G):
32         # Add edges to connect components
33         components = list(nx.connected_components(G))

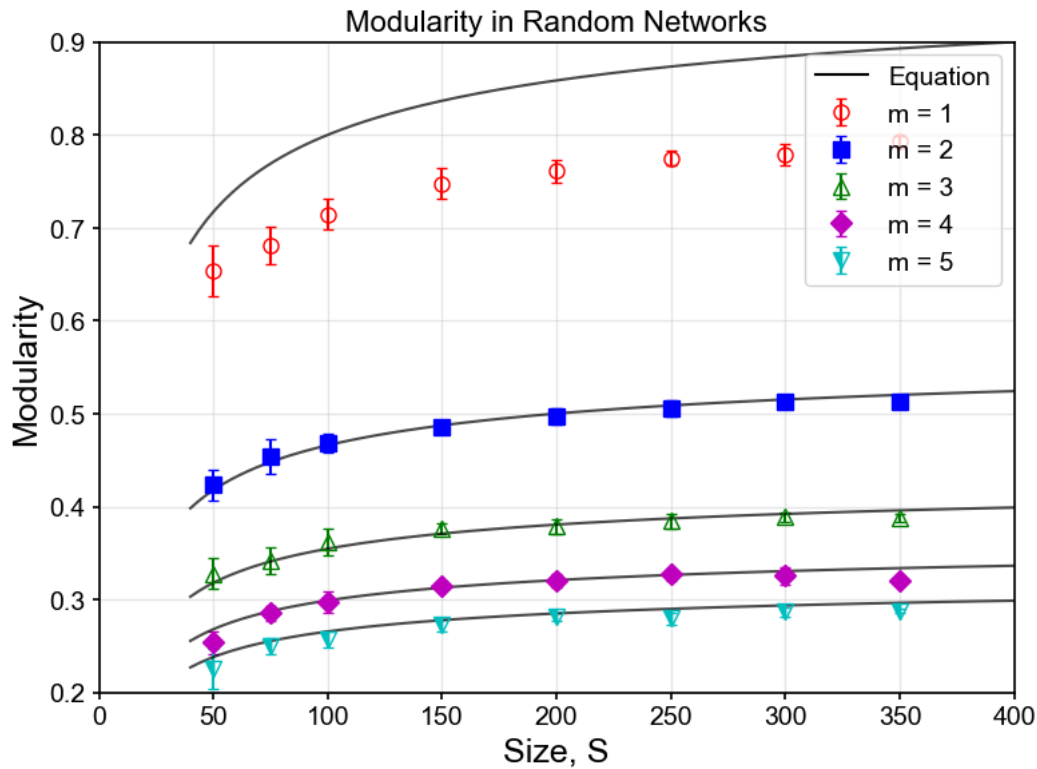
```

```

34         for i in range(len(components) - 1):
35             node1 = list(components[i])[0]
36             node2 = list(components[i + 1])[0]
37             G.add_edge(node1, node2)
38
39     return G
40
41 m_values = [1, 2, 3, 4, 5]
42 markers = ['o', 's', '^', 'D', 'v']
43 sizes = [50, 75, 100, 150, 200, 250, 300, 350]
44 num_trials = 10 # Number of trials for each configuration
45 colors = ['r', 'b', 'g', 'm', 'c']
46 fillstyles = ['none', 'full', 'none', 'full', 'left']
47
48 # Store results
49 results = {m: {'sizes': [], 'modularity_mean': [], 'modularity_std': []} for m in m_values}
50
51 # Calculate modularity for each configuration
52 for m in m_values:
53     print(f"Processing m = {m}")
54     for size in sizes:
55         modularities = []
56         for trial in range(num_trials):
57             G = generate_random_graph(size, m)
58             mod = calculate_modularity(G)
59             modularities.append(mod)
60
61         results[m]['sizes'].append(size)
62         results[m]['modularity_mean'].append(np.mean(modularities))
63         results[m]['modularity_std'].append(np.std(modularities))
64
65 # Create the plot
66 plt.figure(figsize=(8, 6))
67
68 # Plot data points and theoretical curves
69 for i, m in enumerate(m_values):
70     sizes = np.array(results[m]['sizes'])
71     means = np.array(results[m]['modularity_mean'])
72     stds = np.array(results[m]['modularity_std'])
73
74     # Plot data points with error bars
75     plt.errorbar(sizes, means, yerr=stds,
76                 marker=markers[i], markersize=8,
77                 fillstyle=fillstyles[i], color=colors[i],
78                 linestyle='none', capsize=3,
79                 label=f'm = {m}')
80
81     # Plot theoretical curves
82     sizes_theory = np.linspace(40, 400, 100)
83     # Adjust the theoretical curve to better match random graph behavior
84     modularity_theory = theoretical_curve(sizes_theory, m, a=0.165)
85     modularity_theory = modularity_theory
86     plt.plot(sizes_theory, modularity_theory, 'k-', linewidth=1.5, alpha=0.7)
87
88 # Add the legend entry for Equation
89 plt.plot([], [], 'k-', linewidth=1.5, label='Equation')
90
91 # Formatting
92 plt.xlabel('Size, S', fontsize=18)
93 plt.ylabel('Modularity', fontsize=18)
94 plt.xlim(0, 400)
95 plt.ylim(0.2, 0.9)
96 plt.xticks(fontsize=14)
97 plt.yticks(fontsize=14)
98 plt.grid(True, alpha=0.3)
99
100 # Create custom legend
101 handles, labels = plt.gca().get_legend_handles_labels()
102 legend_order = [0, 1, 2, 3, 4, 5] # Order: m=1 to m=5, then Equation
103 handles = [handles[i] for i in legend_order]
104 labels = [labels[i] for i in legend_order]
105 plt.legend(handles, labels, loc='upper right', fontsize=14, frameon=True)
106
107 plt.title('Modularity in Random Networks', fontsize=16)
108
109 plt.tight_layout()
110 plt.subplots_adjust(bottom=0.12)
111 plt.show()

```

Processing m = 1
 Processing m = 2
 Processing m = 3
 Processing m = 4
 Processing m = 5



For “normal” densities ($m \geq 2$), optimized modularity in random graphs follows almost the same scaling as in scale-free networks, emphasizing that high Q can be a purely random effect.

For very sparse random graphs ($m=1$), the simple ansatz breaks down, highlighting that modularity is also sensitive to sparsity and local structure, not just to “true” communities.

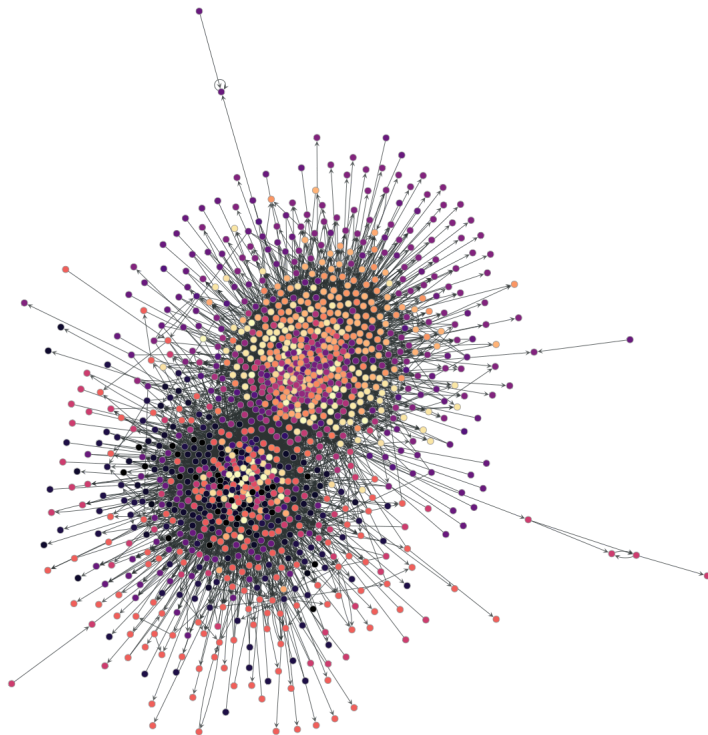
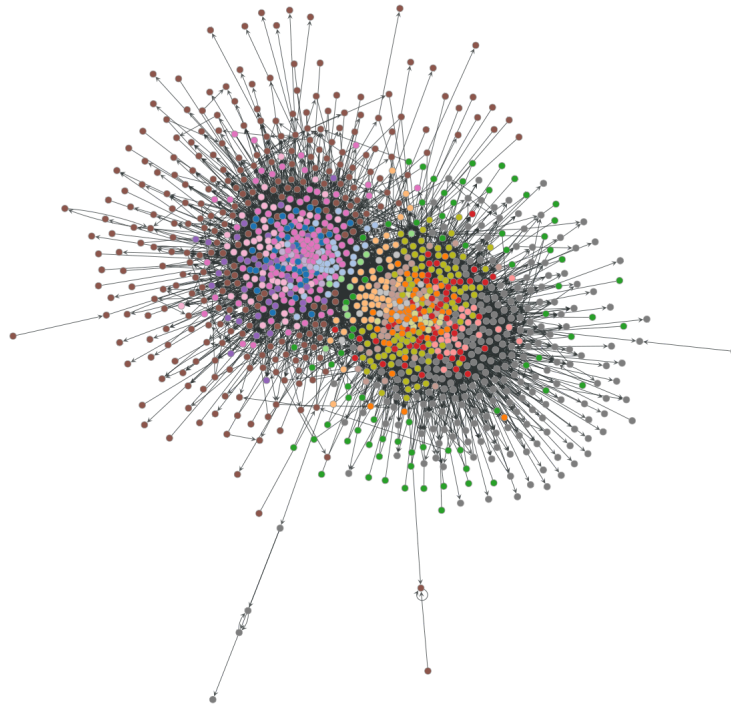
It shows that modularity by itself is a relative and model-dependent measure of partition quality, not an absolute indicator that communities are really present.

~ b)

```

1 g = gt.collection.ns["polblogs"]
2
3 # get the largest connected component
4 g = gt.extract_largest_component(g, directed=False)
5
6 state_dc = gt.minimize_blockmodel_dl(g, state_args={'degree_corrected': True, 'B': 2})
7 state_nodc = gt.minimize_blockmodel_dl(g, state_args={'degree_corrected': False, 'B': 2})
8 gt.graph_draw(g, vertex_fill_color=state_dc.get_blocks(), output_size=(600, 600), output=None)
9 gt.graph_draw(g, vertex_fill_color=state_nodc.get_blocks(), output_size=(600, 600), output=None)

```



<VertexPropertyMap object with value type 'vector<double>', for Graph 0x1c74f3710, at 0x1c7543980>

✓ c)

```
1 # now, print the description length of the degree-corrected and non-degree-corrected models above
2 print("Description Length (Degree-Corrected):", state_dc.entropy())
3 print("Description Length (Non-Degree-Corrected):", state_nodc.entropy())
```

```
Description Length (Degree-Corrected): 61086.12386979071
Description Length (Non-Degree-Corrected): 65100.71088702207
```

```
1 # g is the largest connected component of the polblogs dataset
2 dc_state_new = gt.minimize_blockmodel_dl(g, state_args={'degree_corrected': True})
3 nodc_state_new = gt.minimize_blockmodel_dl(g, state_args={'degree_corrected': False})
4
5 desc_lengths_dc = []
6 desc_lengths_nodc = []
7 for _ in range(1000):
```

```

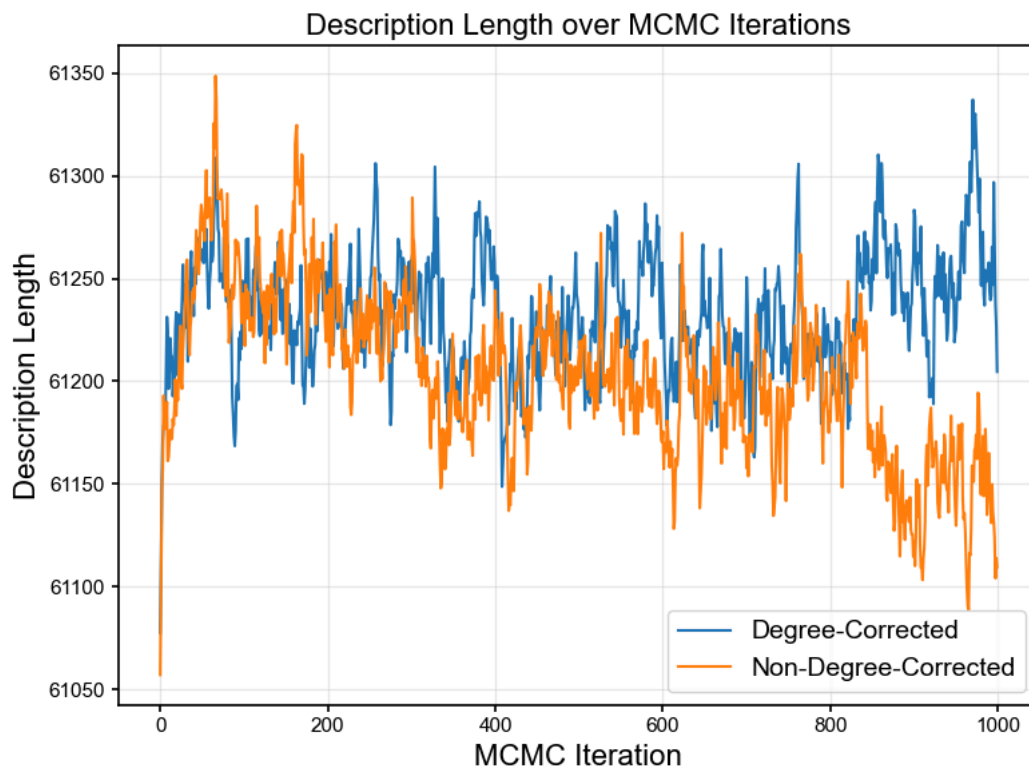
8     dc_state_new.mcmc_sweep(niter=1)
9     nodc_state_new.mcmc_sweep(niter=1)
10    desc_lengths_dc.append(dc_state_new.entropy())
11    desc_lengths_nodc.append(nodc_state_new.entropy())

```

```

1 # Visualize both of these curves in a single plot, labeled properly, and comment on your observation.
2 plt.figure(figsize=(8, 6))
3 plt.plot(desc_lengths_dc, label='Degree-Corrected', color='#1f77b4')
4 plt.plot(desc_lengths_nodc, label='Non-Degree-Corrected', color='#ff7f0e')
5 plt.xlabel('MCMC Iteration', fontsize=16)
6 plt.ylabel('Description Length', fontsize=16)
7 plt.title('Description Length over MCMC Iterations', fontsize=16)
8 plt.legend(fontsize=14)
9 plt.grid(True, alpha=0.3)
10 plt.tight_layout()
11 plt.show()

```



✓ d)

Yes. The Figure 7 in the paper shows that for DC (degree-corrected) SBM, the two communities are completely distinguished, while in the SBM (standard SBM) case, the communities are mixed together. This indicates that degree correction helps to better identify the underlying community structure in networks with heterogeneous degree distributions.

As it was mentioned in <https://graph-tool.skewed.de/static/docs/stable/demos/inference/inference.html#model-selection>, the degree-corrected SBM accounts for the degree heterogeneity of nodes within communities, allowing for a more accurate representation of real-world networks where node degrees can vary significantly. This leads to improved community detection performance compared to the standard SBM, which assumes uniform degree distributions within communities.

To Summarize, we can think of degree correction as a way to account for the fact that nodes in real-world networks often have varying degrees (number of connections). In the standard SBM, nodes within the same community are assumed to have similar connection patterns, which can lead to inaccuracies when nodes have significantly different degrees.

✓ Question 2

✓ a)

I'll do it for one of the most common algorithms of dimension reduction, PCA. I'll start with a 3D dataset and reduce it to 2D, and then reduce it to 1D.

What is PCA? Principal Component Analysis (PCA) is a statistical technique used for dimensionality reduction while preserving as much variance as possible in the data. It transforms the original variables into a new set of uncorrelated variables called principal

components, ordered by the amount of variance they capture from the data. Imagine you have a huge dataset with many features (dimensions). Visualizing or analyzing such high-dimensional data can be challenging and computationally expensive. PCA helps by reducing the number of dimensions while retaining the most important information. For example, let's say you work with a dataset containing information about various fruits, including features like weight, volume, color, time of harvest, sweetness level, sourness, nutrient content (which itself could be multiple features), and more. PCA can help you reduce these multiple features into a smaller set of principal components that still capture the essence of the data. The final features could be combinations of the original features, such as a "size" component (combining weight and volume) and a "taste" component (combining sweetness and sourness). The final feature could also be an already existing feature like nutrient content. The goal is to simplify the dataset while retaining the most important information for analysis or visualization.

To implement PCA, we follow these steps:

1. Standardize the Data: Center the data by subtracting the mean of each feature and scale it to have unit variance.
2. Compute the Covariance Matrix: Calculate the covariance matrix to understand how the features vary with each other.
3. Eigenvalue Decomposition: Compute the eigenvalues and eigenvectors of the covariance matrix. The eigenvectors represent the directions of maximum variance (principal components), and the eigenvalues indicate the amount of variance captured by each principal component.
4. Sort and Select Components: Sort the eigenvalues in descending order and select the top k eigenvectors corresponding to the largest eigenvalues. These eigenvectors form the new feature space.
5. Transform the Data: Project the original data onto the new feature space by multiplying the standardized data with the selected eigenvectors.

```
1 # build a pedagogically effective visualization of the concept of dimensionality reduction using PCA on a sy
2 # import necessary libraries
3 from sklearn.decomposition import PCA
4 from sklearn.datasets import make_blobs
5 import matplotlib.pyplot as plt
6 import numpy as np
7
8 # Set a style for better aesthetics
9 plt.style.use('seaborn-v0_8-whitegrid')
10 sns.set_palette('viridis')
11 # generate synthetic dataset in 3 dimensions
12 X, y = make_blobs(n_samples=300, centers=4, cluster_std=1.0, n_features=2, random_state=42)
13 # visualize original data in 3D
14 fig = plt.figure(figsize=(8, 6))
15 ax = fig.add_subplot(111, projection='3d')
16 scatter = ax.scatter(X[:, 0], X[:, 1], c=y, cmap='viridis', s=50)
17 ax.set_title('Original Data in 3D', fontsize=16)
18 ax.set_xlabel('Feature 1', fontsize=14)
19 ax.set_ylabel('Feature 2', fontsize=14)
20 ax.set_zlabel('Feature 3', fontsize=14)
21 ax.view_init(elev=20, azim=130) # Set a nice viewing angle
22 fig.colorbar(scatter, ax=ax, label='Cluster ID')
23 plt.tight_layout()
24 plt.show()
25
26 # apply PCA to reduce dimensionality from 3D to 2D
27 pca = PCA(n_components=2)
28 X_reduced = pca.fit_transform(X)
29 # visualize reduced data in 2D
30 plt.figure(figsize=(6, 6))
31 plt.scatter(X_reduced[:, 0], X_reduced[:, 1], c=y, cmap='viridis', s=50)
32 plt.title('Data after PCA Reduction to 2D', fontsize=16)
33 plt.xlabel('Principal Component 1', fontsize=14)
34 plt.ylabel('Principal Component 2', fontsize=14)
35 plt.grid(True, alpha=0.3)
36 plt.tight_layout()
37 plt.show()
38 # apply PCA to reduce dimensionality from 2D to 1D
39 pca_1d = PCA(n_components=1)
40 X_reduced_1d = pca_1d.fit_transform(X_reduced)
41 # visualize reduced data in 1D
42 plt.figure(figsize=(5, 5))
43 plt.scatter(X_reduced_1d, np.zeros_like(X_reduced_1d), c=y, cmap='viridis', s=50)
44 plt.title('Data after PCA Reduction to 1D', fontsize=16)
45 plt.xlabel('Principal Component 1', fontsize=14)
46 plt.yticks([])
47 plt.grid(True, alpha=0.3)
48 plt.tight_layout()
49 plt.show()
```


Original Data in 3D

b)

The aspect of PCA that I find most confusing is the interpretation of principal components. While I understand that they are linear combinations of the original features, it can be challenging to really understand what these new dimensions represent in a real-world context. For example, if we reduce a dataset of fruits to two principal components, it can be difficult to interpret what these components mean in terms of actual fruit characteristics.

Another kind of confusing aspect is the reason we need to use eigenvalues and eigenvectors. I understand that they help us identify the directions of maximum variance, but the mathematical process behind it can be a bit abstract. A few years ago I used to dig deeper into the linear algebra behind PCA to better understand why eigenvalues and eigenvectors are important; but I don't honestly remember the details now.

My understanding:

About my understanding, I think the component interpretations could have a deep meaning in terms of feature combinations, while it may not be humanly interpretable. It's similar to a Convolutional Neural Network (CNN) where the convolutional layers extract features that may not have a direct human interpretation but are useful for the model's performance. I remember a simple CNN that inputs images of handwritten digits and outputs the digit class. The first layer extracted colors, then edges, and then more complex shapes that we didn't quite understand as human beings, but have a mathematical interpretation. So the same could be true for PCA, where the principal components may not have a direct human interpretation but are useful for capturing the variance in the data.

About the eigenvalues and eigenvectors, let's write down the algebra behind it: Let's have a Matrix X of shape $(n_{\text{samples}}, n_{\text{features}})$, where each row represents a data point and each column represents a feature. Assume we did the preprocess and already standardized the data. Now we want to get the covariance matrix C of shape $(n_{\text{features}}, n_{\text{features}})$:

$$C = (1 / (n_{\text{samples}} - 1)) * X.T @ X$$

This C matrix tells us how the features vary with each other. Now we want to do the eigenvalue decomposition of the covariance matrix C :

$$\text{eigenvalues, eigenvectors} = \text{np.linalg.eig}(C)$$

This is a numpy function that computes the eigenvalues and eigenvectors of the matrix C . The real calculation behind it is solving the equation:

$$C * v = \lambda * v$$

where C is the covariance matrix, v is the eigenvector, and λ (lambda) is the eigenvalue. This equation states that when we multiply the covariance matrix C by an eigenvector v , the result is simply a scaled version of that eigenvector (scaled by the eigenvalue λ). The eigenvectors represent the directions of maximum variance in the data, and the eigenvalues indicate the amount of variance captured by each eigenvector. By selecting the top k eigenvectors corresponding to the largest eigenvalues, we can form a new feature space that captures the most important information in the data.

c)

```
1 import matplotlib.pyplot as plt
2 import matplotlib.patches as mpatches
3 from sklearn.decomposition import PCA
4 from sklearn.datasets import make_blobs
5 import numpy as np
6 from matplotlib.patches import FancyBboxPatch, ConnectionPatch
7 from matplotlib.offsetbox import AnchoredText
8 import matplotlib.gridspec as gridspec
9
10 # Set random seed for reproducibility
11 np.random.seed(42)
12
13 fig = plt.figure(figsize=(16, 9), facecolor='#f8f9fa')
14
15 main_gs = gridspec.GridSpec(2, 1, height_ratios=[1, 2.5], hspace=0.15, figure=fig)
16
17 concept_gs = gridspec.GridSpecFromSubplotSpec(1, 4, subplot_spec=main_gs[0],
18                                             width_ratios=[1, 1.2, 1, 1], wspace=0.05)
19
```

```

20 title_ax = fig.add_subplot(concept_gs[0])
21 title_ax.axis('off')
22 title_ax.text(0.5, 0.7, 'Principal Component\nAnalysis (PCA)',
23              fontsize=20, fontweight='bold', ha='center', va='center',
24              color='#2c3e50')
25 title_ax.text(0.5, 0.3, 'Dimensionality Reduction\nThrough Variance Maximization',
26              fontsize=11, ha='center', va='center', color='#7f8c8d', style='italic')
27
28 concept_ax = fig.add_subplot(concept_gs[1])
29 concept_ax.axis('off')
30
31 concept_text = (
32     "WHAT IS PCA?\n\n"
33     "* Transforms correlated variables into\n"
34     "  uncorrelated principal components\n\n"
35     "* Preserves maximum variance in data\n\n"
36     "* Reduces computational complexity\n\n"
37     "* Enables visualization of high-D data"
38 )
39
40 bbox_props = dict(boxstyle="round,pad=0.5", facecolor='#e8f4f8',
41                  edgecolor='#3498db', linewidth=2)
42 concept_ax.text(0.05, 0.5, concept_text, transform=concept_ax.transAxes,
43               fontsize=10, verticalalignment='center', bbox=bbox_props)
44
45 # Mathematical Formulation
46 math_ax = fig.add_subplot(concept_gs[2])
47 math_ax.axis('off')
48
49 math_text = (
50     "MATHEMATICAL FOUNDATION\n\n"
51     "1. Standardize:  $\tilde{X} = X - \mu$ \n\n"
52     "2. Covariance:  $C = (1/n-1)\tilde{X}^T\tilde{X}$ \n\n"
53     "3. Eigendecomposition:  $Cv = \lambda v$ \n\n"
54     "4. Transform:  $Y = \tilde{X}W$ "
55 )
56
57 math_bbox = dict(boxstyle="round,pad=0.5", facecolor='#fff4e6',
58                 edgecolor='#f39c12', linewidth=2)
59 math_ax.text(0.05, 0.5, math_text, transform=math_ax.transAxes,
60             fontsize=10, verticalalignment='center', bbox=math_bbox)
61
62 # Steps Box
63 steps_ax = fig.add_subplot(concept_gs[3])
64 steps_ax.axis('off')
65
66 steps_text = (
67     "PCA STEPS\n\n"
68     "1. Center and scale data\n\n"
69     "2. Compute covariance matrix\n\n"
70     "3. Find eigenvectors & values\n\n"
71     "4. Select top k components\n\n"
72     "5. Project onto new space"
73 )
74
75 steps_bbox = dict(boxstyle="round,pad=0.5", facecolor='#f0f8ff',
76                  edgecolor='#9b59b6', linewidth=2)
77 steps_ax.text(0.05, 0.5, steps_text, transform=steps_ax.transAxes,
78             fontsize=10, verticalalignment='center', bbox=steps_bbox)
79
80 viz_gs = gridspec.GridSpecFromSubplotSpec(2, 5, subplot_spec=main_gs[1],
81                                           height_ratios=[1.5, 1], width_ratios=[1, 1, 1, 1, 1],
82                                           wspace=0.15, hspace=0.20)
83
84 # Generate synthetic 3D dataset
85 X, y = make_blobs(n_samples=300, centers=3, cluster_std=0.8, n_features=3, random_state=42)
86
87 # Apply PCA
88 pca_3d = PCA(n_components=3)
89 X_normalized = (X - X.mean(axis=0)) / X.std(axis=0)
90 pca_3d.fit(X_normalized)
91
92 pca_2d = PCA(n_components=2)
93 X_2d = pca_2d.fit_transform(X_normalized)
94
95 pca_1d = PCA(n_components=1)
96 X_1d = pca_1d.fit_transform(X_normalized)
97
98 # --- Plot 1: Original 3D Data with PC directions ---
99 ax1 = fig.add_subplot(viz_gs[0, 0:2], projection='3d')
100 scatter1 = ax1.scatter(X_normalized[:, 0], X_normalized[:, 1], X_normalized[:, 2],
101                       c=y, cmap='viridis', s=25, alpha=0.6, edgecolors='w', linewidth=0.5)

```

```

102
103 # Add principal component directions
104 origin = np.mean(X_normalized, axis=0)
105 for i, (vec, val) in enumerate(zip(pca_3d.components_, pca_3d.explained_variance_)):
106     ax1.quiver(origin[0], origin[1], origin[2],
107               vec[0]*2, vec[1]*2, vec[2]*2,
108               color=['red', 'blue', 'green'][i], arrow_length_ratio=0.1,
109               linewidth=2.5, alpha=0.8, label=f'PC{i+1}')
110
111 ax1.set_title('Original 3D Data\nwith Principal Components', fontsize=11, fontweight='bold')
112 ax1.set_xlabel('X1', fontsize=9)
113 ax1.set_ylabel('X2', fontsize=9)
114 ax1.set_zlabel('X3', fontsize=9)
115 ax1.view_init(elev=20, azim=45)
116 ax1.legend(loc='upper left', fontsize=7)
117
118 # --- Plot 2: 2D Projection ---
119 ax2 = fig.add_subplot(viz_gs[0, 2])
120 scatter2 = ax2.scatter(X_2d[:, 0], X_2d[:, 1], c=y, cmap='viridis',
121                       s=25, alpha=0.6, edgecolors='w', linewidth=0.5)
122
123 # Add arrows showing PC directions
124 ax2.arrow(0, 0, 2, 0, head_width=0.1, head_length=0.1,
125          fc='red', ec='red', alpha=0.8, linewidth=2)
126 ax2.arrow(0, 0, 0, 2, head_width=0.1, head_length=0.1,
127          fc='blue', ec='blue', alpha=0.8, linewidth=2)
128 ax2.text(2.2, 0, 'PC1', fontsize=9, fontweight='bold', color='red')
129 ax2.text(0, 2.2, 'PC2', fontsize=9, fontweight='bold', color='blue')
130
131 ax2.set_title(f'2D Projection\n({pca_2d.explained_variance_ratio_.sum():.1%} variance)',
132             fontsize=11, fontweight='bold')
133 ax2.set_xlabel('First PC', fontsize=9)
134 ax2.set_ylabel('Second PC', fontsize=9)
135 ax2.grid(True, alpha=0.3, linestyle='--')
136 ax2.set_xlim([-3, 3])
137 ax2.set_ylim([-3, 3])
138
139 # --- Plot 3: 1D Projection ---
140 ax3 = fig.add_subplot(viz_gs[0, 3])
141 # Create jitter for better visualization
142 jitter = np.random.normal(0, 0.02, size=len(X_1d))
143 scatter3 = ax3.scatter(X_1d, jitter, c=y, cmap='viridis',
144                      s=25, alpha=0.6, edgecolors='w', linewidth=0.5)
145
146 ax3.set_title(f'1D Projection\n({pca_1d.explained_variance_ratio_[0]:.1%} variance)',
147             fontsize=11, fontweight='bold')
148 ax3.set_xlabel('First PC', fontsize=9)
149 ax3.set_ylim([-0.5, 0.5])
150 ax3.set_yticks([])
151 ax3.grid(True, alpha=0.3, linestyle='--', axis='x')
152 ax3.axhline(y=0, color='red', linestyle='-', alpha=0.5, linewidth=2)
153
154 # --- Plot 4: Variance Explained ---
155 ax4 = fig.add_subplot(viz_gs[0, 4])
156 explained_var = pca_3d.explained_variance_ratio_
157 cumulative_var = np.cumsum(explained_var)
158
159 x_pos = np.arange(1, 4)
160 bars = ax4.bar(x_pos, explained_var, color=['#e74c3c', '#3498db', '#2ecc71'],
161               alpha=0.7, edgecolor='black', linewidth=1.5, label='Individual')
162 ax4.plot(x_pos, cumulative_var, 'o-', color='#34495e', linewidth=2,
163         markersize=6, label='Cumulative')
164
165 # Add percentage labels
166 for i, (bar, var, cum) in enumerate(zip(bars, explained_var, cumulative_var)):
167     ax4.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
168             f'{var:.1%}', ha='center', fontsize=8, fontweight='bold')
169     ax4.text(i+1, cum + 0.03, f'{cum:.1%}', ha='center', fontsize=8, color='#34495e')
170
171 ax4.set_title('Variance Explained', fontsize=11, fontweight='bold')
172 ax4.set_xlabel('Component', fontsize=9)
173 ax4.set_ylabel('Proportion', fontsize=9)
174 ax4.set_xticks(x_pos)
175 ax4.set_xticklabels(['PC1', 'PC2', 'PC3'])
176 ax4.set_ylim([0, 1.2])
177 ax4.legend(loc='upper right', fontsize=8)
178 ax4.grid(True, alpha=0.3, linestyle='--', axis='y')
179
180 # --- Reconstruction Error Visualization ---
181 ax5 = fig.add_subplot(viz_gs[1, 0:2])
182
183 # Reconstruct data from different numbers of components

```

```

184 errors = []
185 n_components_range = range(1, 4)
186 for n in n_components_range:
187     pca_temp = PCA(n_components=n)
188     X_transformed = pca_temp.fit_transform(X_normalized)
189     X_reconstructed = pca_temp.inverse_transform(X_transformed)
190     error = np.mean((X_normalized - X_reconstructed) ** 2)
191     errors.append(error)
192
193 ax5.plot(n_components_range, errors, 'o-', color='#e74c3c', linewidth=2, markersize=6)
194 ax5.fill_between(n_components_range, errors, alpha=0.3, color='#e74c3c')
195 ax5.set_title('Reconstruction Error', fontsize=11, fontweight='bold')
196 ax5.set_xlabel('Number of Components', fontsize=9)
197 ax5.set_ylabel('Mean Squared Error', fontsize=9)
198 ax5.set_xticks(n_components_range)
199 ax5.grid(True, alpha=0.3, linestyle='--')
200
201 # --- Data Distribution Before/After PCA ---
202 ax6 = fig.add_subplot(viz_gs[1, 2:5])
203
204 # Calculate correlation matrices
205 corr_before = np.corrcoef(X_normalized.T)
206 corr_after = np.corrcoef(X_2d.T)
207
208 # Create subplots for correlation matrices
209 im1 = ax6.imshow(corr_before, cmap='coolwarm', aspect='auto', vmin=-1, vmax=1)
210 ax6.set_title('Feature Correlation Before PCA', fontsize=11, fontweight='bold')
211 ax6.set_xticks([0, 1, 2])
212 ax6.set_yticks([0, 1, 2])
213 ax6.set_xticklabels(['X1', 'X2', 'X3'], fontsize=9)
214 ax6.set_yticklabels(['X1', 'X2', 'X3'], fontsize=9)
215
216 # Add correlation values
217 for i in range(3):
218     for j in range(3):
219         text = ax6.text(j, i, f'{corr_before[i, j]:.2f}',
220                        ha="center", va="center", color="black" if abs(corr_before[i, j]) < 0.5 else "white",
221                        fontsize=9)
222
223 # Add colorbar
224 plt.colorbar(im1, ax=ax6, fraction=0.046, pad=0.04)
225
226 # Add flow arrows between plots
227 arrow1 = ConnectionPatch((1, 0.5), (0, 0.5), "axes fraction", "axes fraction",
228                        axesA=ax1, axesB=ax2, color='gray', arrowstyle='->',
229                        connectionstyle="arc3,rad=0.1", linewidth=2, alpha=0.6)
230 fig.add_artist(arrow1)
231
232 arrow2 = ConnectionPatch((1, 0.5), (0, 0.5), "axes fraction", "axes fraction",
233                        axesA=ax2, axesB=ax3, color='gray', arrowstyle='->',
234                        connectionstyle="arc3,rad=0.1", linewidth=2, alpha=0.6)
235 fig.add_artist(arrow2)
236
237 # Add key takeaway box at the bottom
238 takeaway_text = (
239     "KEY TAKEAWAY: PCA finds the optimal linear transformation that maximizes variance retention "
240     "while reducing dimensionality. Perfect for visualization, noise reduction, and feature extraction!"
241 )
242 fig.text(0.5, 0.02, takeaway_text, ha='center', fontsize=11,
243         bbox=dict(boxstyle="round,pad=0.4", facecolor='ffffcc', edgecolor='#f39c12', linewidth=2))
244
245 plt.suptitle('', fontsize=1) # Remove any default title
246 plt.tight_layout()
247 plt.show()

```

Principal Component Analysis (PCA)

*Dimensionality Reduction
Through Variance Maximization*

WHAT IS PCA?

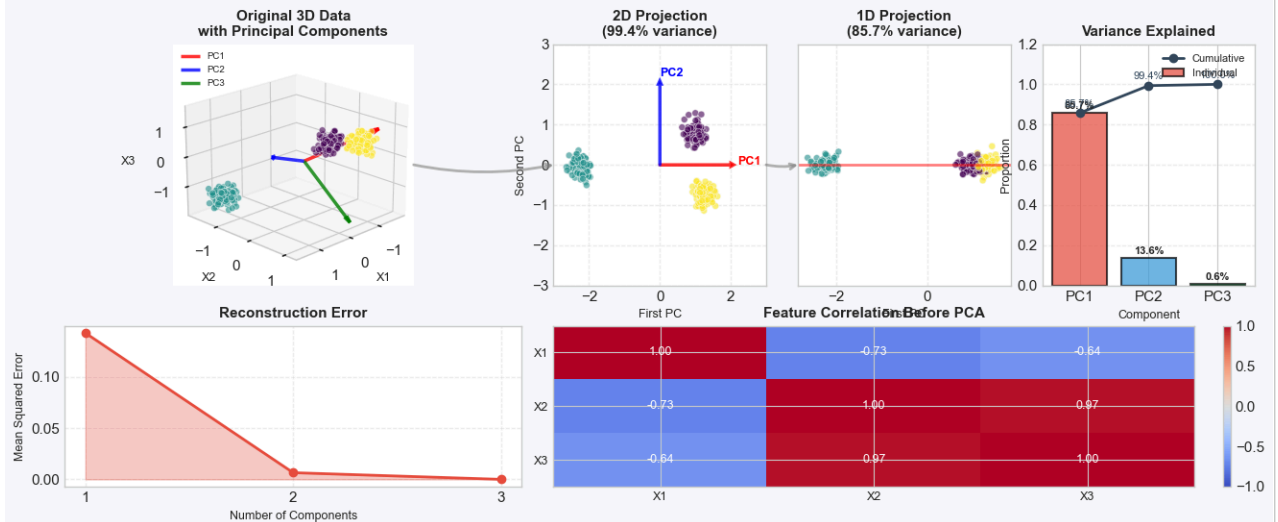
- * Transforms correlated variables into uncorrelated principal components
- * Preserves maximum variance in data
- * Reduces computational complexity
- * Enables visualization of high-D data

MATHEMATICAL FOUNDATION

1. Standardize: $X = X - \mu$
2. Covariance: $C = (1/n-1)X^T X$
3. Eigendecomposition: $Cv = \lambda v$
4. Transform: $Y = XW$

PCA STEPS

1. Center and scale data
2. Compute covariance matrix
3. Find eigenvectors & values
4. Select top k components
5. Project onto new space



KEY TAKEAWAY: PCA finds the optimal linear transformation that maximizes variance retention while reducing dimensionality. Perfect for visualization, noise reduction, and feature extraction!

Question 3

a)

```
1 def compute_commute_time(G, i, j):
2     """
3     Compute the commute time between two nodes i and j in an
4     undirected, connected graph G using the Laplacian pseudoinverse.
5
6     Commute time C_ij = vol(G) * (L^+_ii + L^+_jj - 2 * L^+_ij),
7     where L^+ is the Moore-Penrose pseudoinverse of the Laplacian.
8     """
9     # Fix a node ordering
10    nodes = list(G.nodes())
11    idx = {node: k for k, node in enumerate(nodes)}
12
13    # Sanity check
14    if i not in idx or j not in idx:
15        raise ValueError("Both i and j must be nodes of G")
16
17    # Combinatorial Laplacian as a dense NumPy array
18    L_sparse = nx.laplacian_matrix(G, nodelist=nodes)
19    # For newer scipy, L_sparse is a csr_array with .toarray()
20    L = L_sparse.toarray().astype(float)
21
22    # Pseudoinverse of L
23    L_pinv = np.linalg.pinv(L)
24
25    # Volume of the graph = sum of degrees (uses weights if present)
26    vol = sum(d for _, d in G.degree(weight="weight"))
27
28    ii, jj = idx[i], idx[j]
29    C_ij = vol * (L_pinv[ii, ii] + L_pinv[jj, jj] - 2.0 * L_pinv[ii, jj])
30    return C_ij
31
```

```
32
33 # Load the Karate club graph
34 G = nx.karate_club_graph()
35
36 # Pick three random unordered node pairs
37 random.seed(7332) # for reproducibility
38 nodes = list(G.nodes())
39 all_pairs = [(u, v) for k, u in enumerate(nodes) for v in nodes[k+1:]]
40 pairs = random.sample(all_pairs, 10)
41
42 print("Commute times on the Karate club graph:")
43 differences = []
44 for (u, v) in pairs:
45     c = compute_commute_time(G, u, v)
46     print(f"Pair ({u}, {v}): commute time = {c:.4f}")
47     # calculate the expected commute time using the formula
48     deg_u = G.degree(u)
49     deg_v = G.degree(v)
50     m = G.number_of_edges()
51     expected_c = 2 * m * (1/deg_u + 1/deg_v)
52     print(f"Pair ({u}, {v}): expected commute time = {expected_c:.4f}\n")
53     differences.append(abs(c - expected_c))
54
55 # plot the histogram of differences between computed and expected commute times
56 plt.figure(figsize=(8, 5))
57 plt.hist(differences, bins=10, color='#3498db', edgecolor='black', alpha=0.7)
58 plt.title('Histogram of Differences between Computed and Expected Commute Times', fontsize=16)
59 plt.xlabel('Absolute Difference', fontsize=14)
60 plt.ylabel('Frequency', fontsize=14)
61 plt.grid(True, alpha=0.3)
62 plt.tight_layout()
63 plt.show()
64
```

```

Commute times on the Karate club graph:
Pair (5, 31): commute time = 128.1605
Pair (5, 31): expected commute time = 65.0000

Pair (0, 15): commute time = 109.8628
Pair (0, 15): expected commute time = 87.7500

Pair (7, 19): commute time = 132.6211
Pair (7, 19): expected commute time = 91.0000

Pair (6, 18): commute time = 266.1807
Pair (6, 18): expected commute time = 117.0000

Pair (19, 26): commute time = 213.3516
Pair (19, 26): expected commute time = 130.0000

Pair (6, 31): commute time = 129.8277
Pair (6, 31): expected commute time = 65.0000

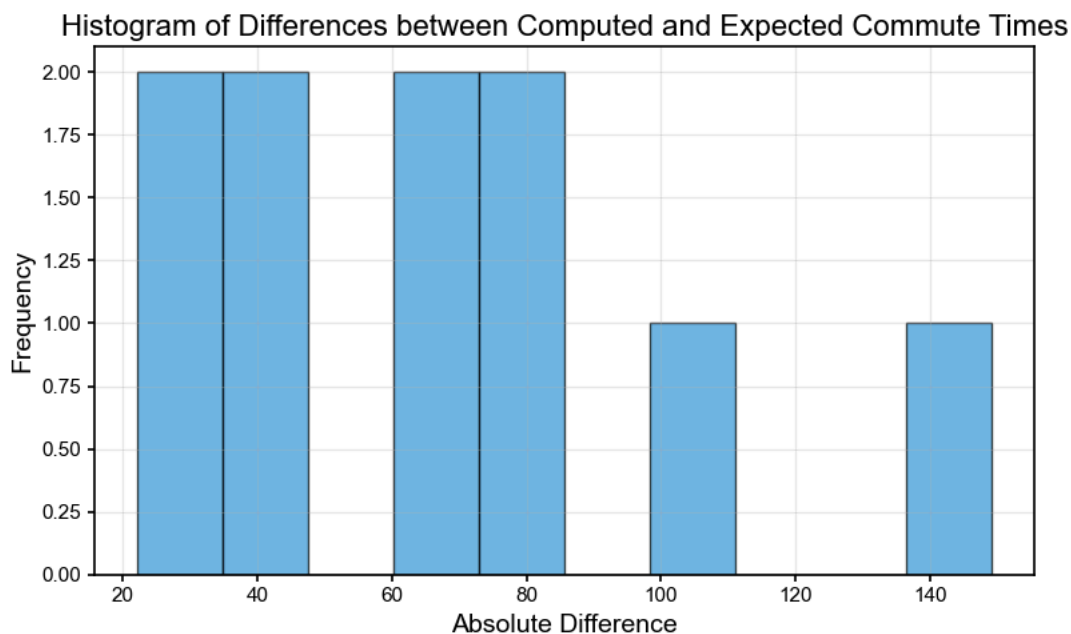
Pair (3, 32): commute time = 69.8669
Pair (3, 32): expected commute time = 39.0000

Pair (2, 4): commute time = 107.8107
Pair (2, 4): expected commute time = 67.6000

Pair (8, 18): commute time = 189.2776
Pair (8, 18): expected commute time = 109.2000

Pair (12, 26): commute time = 261.3807
Pair (12, 26): expected commute time = 156.0000

```



✓ b)

We can do this efficiently using the Laplacian spectrum instead of simulating random walks pair by pair. The commute time $C(u, v)$ between two nodes u and v in a graph can be computed using the formula: $C_{ij} = \text{MFPT}_{ij} + \text{MFPT}_{ji}$ where MFPT_{ij} is the mean first passage time from node i to node j . $C_{ij} = 2M * (L^+_{ii} + L^+_{jj} - 2L^+_{ij})$ where M is the number of edges in the graph, and L^+ is the Moore-Penrose pseudoinverse of the graph Laplacian matrix L .

If we average C_{ij} over all pairs of nodes (i, j) , we get: $\langle C \rangle = 2m / (N - 1) * \text{trace}(L^+)$ where m is the number of edges, N is the number of nodes, and $\text{trace}(L^+)$ is the sum of the diagonal elements of the pseudoinverse of the Laplacian matrix.

```

1 import statistics
2 import math
3
4 def average_commute_time(G):
5     """
6     Return the average commute time over all unordered node pairs (i<j)
7     using the Laplacian pseudoinverse.
8
9     <C> = (2 * vol(G) / (N-1)) * trace(L^+)
10
11     Parameters
12     -----

```

```

13 G : networkx.Graph
14     Undirected, connected graph.
15
16 Returns
17 -----
18 float
19     Average commute time over all node pairs.
20 """
21 # Work on the largest connected component, just to be safe
22 if not nx.is_connected(G):
23     comp = max(nx.connected_components(G), key=len)
24     G = G.subgraph(comp).copy()
25
26 n = G.number_of_nodes()
27 nodes = list(G.nodes())
28
29 # Combinatorial Laplacian
30 L_sparse = nx.laplacian_matrix(G, nodelist=nodes)
31 L = L_sparse.toarray().astype(float)
32
33 # Pseudoinverse
34 L_pinv = np.linalg.pinv(L)
35
36 # trace(L^+)
37 tr_Lp = np.trace(L_pinv)
38
39 # volume of the graph: sum of degrees (== 2M for unweighted graphs)
40 vol = sum(d for _, d in G.degree(weight="weight"))
41
42 # average commute time over all unordered pairs
43 C_avg = 2.0 * vol * tr_Lp / (n - 1)
44 return C_avg
45
46 def er_graph(N=500, k_avg=8, seed=None):
47     """Generate an Erdős-Rényi graph G(n, p) with expected degree k_avg."""
48     p = k_avg / (N - 1)
49     return nx.erdos_renyi_graph(N, p, seed=seed)
50
51 def ba_graph(N=500, m=4, seed=None):
52     """Generate a Barabási-Albert graph with N nodes and m edges per new node."""
53     return nx.barabasi_albert_graph(N, m, seed=seed)
54
55 N=500
56 k_avg=8
57 m=4
58 samples=10
59 base_seed=7332
60
61 er_values = []
62 ba_values = []
63
64 # ER ensemble
65 for s in range(samples):
66     G_er = er_graph(N=N, k_avg=k_avg, seed=base_seed + s)
67     C_er = average_commute_time(G_er)
68     er_values.append(C_er)
69     print(f"ER sample {s}: <C> = {C_er:.3f}")
70
71 # BA ensemble
72 for s in range(samples):
73     G_ba = ba_graph(N=N, m=m, seed=base_seed + 1000 + s)
74     C_ba = average_commute_time(G_ba)
75     ba_values.append(C_ba)
76     print(f"BA sample {s}: <C> = {C_ba:.3f}")
77
78 # Ensemble statistics
79 er_mean = statistics.mean(er_values)
80 er_std = statistics.pstdev(er_values) # population std
81 ba_mean = statistics.mean(ba_values)
82 ba_std = statistics.pstdev(ba_values)
83
84 print("\n--- Summary over ensembles ---")
85 print(f"ER(N={N}, <k>≈{k_avg}): mean <C> = {er_mean:.3f}, std = {er_std:.3f}")
86 print(f"BA(N={N}, m={m}): mean <C> = {ba_mean:.3f}, std = {ba_std:.3f}")

```

```

ER sample 0: <C> = 1359.407
ER sample 1: <C> = 1384.464
ER sample 2: <C> = 1384.886
ER sample 3: <C> = 1378.439
ER sample 4: <C> = 1390.369
ER sample 5: <C> = 1365.953
ER sample 6: <C> = 1343.374
ER sample 7: <C> = 1361.744

```

```

ER sample 8: <C> = 1347.110
ER sample 9: <C> = 1354.343
BA sample 0: <C> = 1591.736
BA sample 1: <C> = 1588.867
BA sample 2: <C> = 1579.825
BA sample 3: <C> = 1580.046
BA sample 4: <C> = 1580.121
BA sample 5: <C> = 1593.932
BA sample 6: <C> = 1589.938
BA sample 7: <C> = 1587.679
BA sample 8: <C> = 1579.014
BA sample 9: <C> = 1592.411

--- Summary over ensembles ---
ER(N=500, <k>≈8): mean <C> = 1367.009, std = 15.818
BA(N=500, m=4):    mean <C> = 1586.357, std = 5.649

```

We saw that the commute times in the BA model are a bit larger than those in the ER model with the same number of nodes and edges. This indicates that, in terms of random-walk dynamics, ER graphs are more efficient for transport. The reason is that ER networks distribute edges more homogeneously, creating many cycles and alternative paths that lower effective resistance distances between nodes. In contrast, BA networks are highly heterogeneous, with many low-degree nodes attached to a few hubs.

Degree distribution is broad / scale-free: a few hubs, many low-degree "leafy" nodes. Many nodes are effectively dangling off a hub or off short tree-like branches.

So even though shortest-path distances in BA can be small (thanks to hubs), the random-walk notion of distance (resistance / commute time) is strongly impacted by the many low-degree nodes and bottlenecks.

Double-click (or enter) to edit

Question 4

Consider a simple, connected, undirected, unweighted graph G with n nodes and m edges. Let A be the adjacency matrix of G , and D be the diagonal degree matrix where $D(i,i)$ is the degree of node i . The random-walk matrix P is defined as $P = D^{-1} A$.

So that a random walker at node i moves to a neighbor j with probability $1/\deg(i)$.

a)

For a connected, undirected graph, the random walk has a stationary distribution π , where $\pi(i) = \deg(i) / (2m)$ for each node i . Here, $2m$ is the sum of all degrees in the graph, which equals twice the number of edges.

Create an ER graph $G(n,p)$ with a value of p that makes it very likely to be connected (e.g., $p \approx \log n/n$ or higher), and if your graph is not connected, regenerate it until it is.

Implement a simple random walk on this graph:

Choose a starting node i at random.

At each time step, move to a uniformly random neighbor.

```

1 while True:
2     G = nx.erdos_renyi_graph(100, 0.05)
3     if nx.is_connected(G):
4         break
5 A = nx.adjacency_matrix(G)
6 degrees = np.array([d for _, d in G.degree()], dtype=float)
7 degrees[degrees == 0.0] = 1.0
8 D_inv = sp.diags(1.0 / degrees)
9 P = D_inv @ A

```

```

1 # At each time step, move to a uniformly random neighbor.
2 def random_walk(P, start_node, n_steps):
3     n_nodes = P.shape[0]
4     current_node = start_node
5     path = [current_node]
6
7     for _ in range(n_steps):
8         row = P.getrow(current_node).toarray().flatten()
9         next_node = np.random.choice(n_nodes, p=row)
10        path.append(next_node)
11        current_node = next_node
12
13    return path
14

```

```

15 start_node = np.random.randint(0, P.shape[0])
16 n_steps = 50
17 path = random_walk(P, start_node, n_steps)
18 print("Random walk path:", path)

```

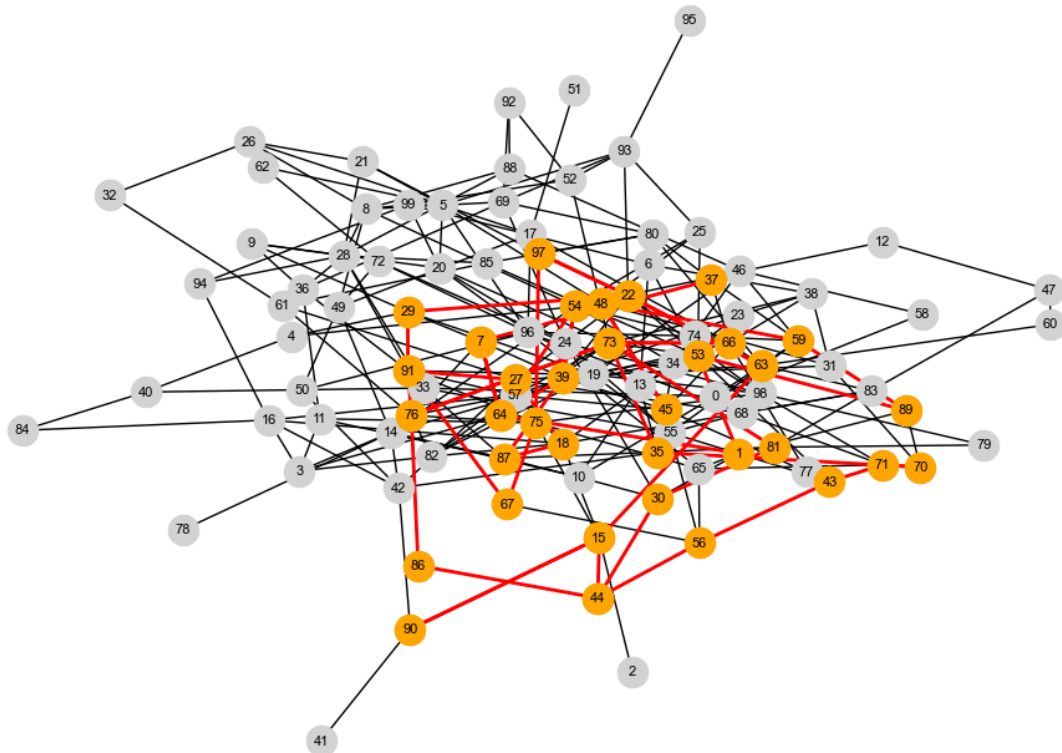
Random walk path: [97, 22, 63, 22, 29, 91, 67, 75, 97, 22, 63, 15, 90, 15, 44, 56, 43, 71, 70, 35, 73, 66, 73, 76]

```

1 plt.figure(figsize=(10, 7))
2 pos = nx.spring_layout(G, seed=42)
3 nx.draw(G, pos, node_color='lightgray', with_labels=True, node_size=300, font_size=8)
4 path_edges = list(zip(path[:-1], path[1:]))
5 nx.draw_networkx_nodes(G, pos, nodelist=path, node_color='orange', node_size=300)
6 nx.draw_networkx_edges(G, pos, edgelist=path_edges, edge_color='red', width=2, arrows=True)
7 plt.title('Random Walk on ER Graph', fontsize=16)
8 plt.show()

```

Random Walk on ER Graph



✓ b)

Run the walk for $T=10^5$ steps (or as large as is reasonable for your machine).

Estimate the empirical stationary distribution π^* by counting how many times the walker visits each node (after discarding an initial burn-in, e.g., first 1000 steps) and normalizing.

```

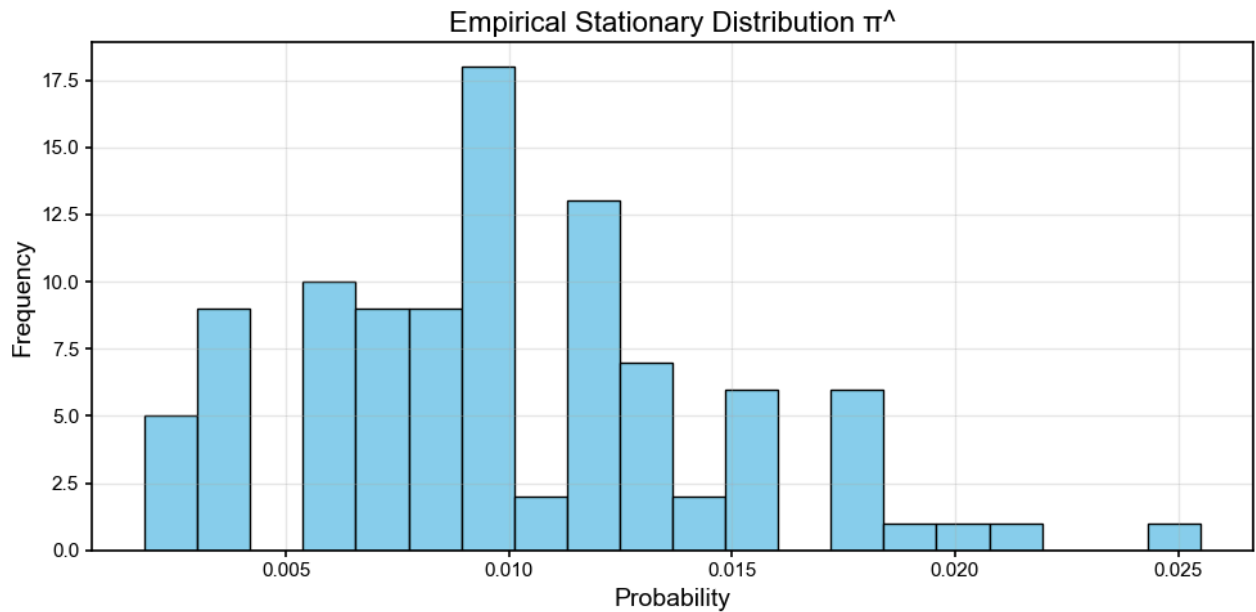
1 T = 500000
2 burn_in = 1000
3 path_long = random_walk(P, start_node, T)
4 visit_counts = np.zeros(G.number_of_nodes())
5 for node in path_long[burn_in:]:
6     visit_counts[node] += 1
7 pi_hat = visit_counts / np.sum(visit_counts)
8 print("Empirical stationary distribution  $\pi^*$ :", pi_hat)

```

Empirical stationary distribution π^* : [0.00752504 0.00995389 0.00210621 0.00961722 0.00601802 0.02164324
0.00780159 0.01132463 0.01386971 0.0077595 0.00976351 0.00783365
0.0039519 0.00982162 0.0156172 0.01202202 0.01127653 0.00974547
0.01575748 0.01741279 0.01174547 0.00772744 0.01554706 0.00576352
0.01169336 0.00778556 0.00783566 0.01365528 0.017519 0.01188174
0.00993184 0.00761121 0.0039018 0.00960519 0.01185769 0.01183164
0.01187974 0.00763125 0.00979357 0.0133627 0.0059579 0.00201002]

```
0.01176952 0.00766331 0.00810018 0.01120839 0.00953706 0.00594788
0.01347292 0.00984768 0.00578155 0.00196392 0.00783566 0.02550095
0.0177675 0.0135631 0.00974146 0.01750898 0.00385971 0.01346691
0.00389378 0.0098116 0.00388777 0.01333865 0.01561921 0.00978756
0.01537071 0.00596993 0.00972944 0.01143284 0.00789778 0.00967934
0.00771742 0.01970136 0.00571742 0.01753503 0.01545087 0.00373747
0.00194188 0.00394989 0.00923245 0.00978756 0.01164126 0.00991782
0.00393987 0.00771141 0.00403005 0.01355308 0.00756712 0.00793385
0.00610219 0.01798794 0.00578356 0.01136871 0.00571542 0.00183166
0.01920036 0.00732864 0.01368334 0.01162723]
```

```
1 # plot the empirical stationary distribution
2 plt.figure(figsize=(10, 5))
3 plt.hist(pi_hat, bins=20, color='skyblue', edgecolor='black')
4 plt.title('Empirical Stationary Distribution  $\pi^{\wedge}$ ', fontsize=16)
5 plt.xlabel('Probability', fontsize=14)
6 plt.ylabel('Frequency', fontsize=14)
7 plt.grid(True, alpha=0.3)
8 plt.tight_layout()
9 plt.show()
```



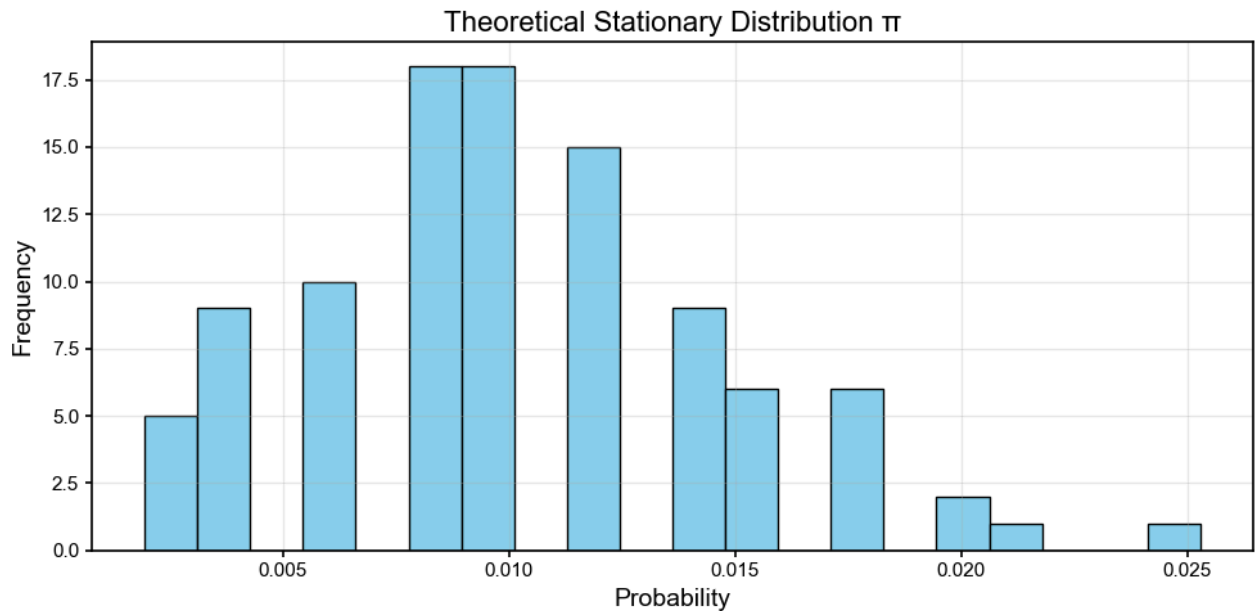
✓ c)

Compute the theoretical stationary distribution π by using the degree distribution of the graph (i.e., $\pi_i = \text{degree}(i) / (2 * \text{number_of_edges})$).

Plot the empirical stationary distribution $\pi^{\wedge}$ versus the theoretical stationary distribution π (e.g., scatter plot) and comment briefly on how close they are and what will happen as T increases.

```
1 theoretical_pi = degrees / np.sum(degrees)
2 print("Theoretical stationary distribution  $\pi$ :", theoretical_pi)
3
4 plt.figure(figsize=(10, 5))
5 plt.hist(theoretical_pi, bins=20, color='skyblue', edgecolor='black')
6 plt.title('Theoretical Stationary Distribution  $\pi$ ', fontsize=16)
7 plt.xlabel('Probability', fontsize=14)
8 plt.ylabel('Frequency', fontsize=14)
9 plt.grid(True, alpha=0.3)
10 plt.tight_layout()
11 plt.show()
```

```
Theoretical stationary distribution  $\pi$ : [0.0077821 0.00972763 0.00194553 0.00972763 0.00583658 0.02140078
0.0077821 0.01167315 0.01361868 0.0077821 0.00972763 0.0077821
0.00389105 0.00972763 0.0155642 0.01167315 0.01167315 0.00972763
0.0155642 0.01750973 0.01167315 0.0077821 0.0155642 0.00583658
0.01167315 0.0077821 0.0077821 0.01361868 0.01750973 0.01167315
0.00972763 0.0077821 0.00389105 0.00972763 0.01167315 0.01167315
0.01167315 0.0077821 0.00972763 0.01361868 0.00583658 0.00194553
0.01167315 0.0077821 0.0077821 0.01167315 0.00972763 0.00583658
0.01361868 0.00972763 0.00583658 0.00194553 0.0077821 0.02529183
0.01750973 0.01361868 0.00972763 0.01750973 0.00389105 0.01361868
0.00389105 0.00972763 0.00389105 0.01361868 0.0155642 0.00972763
0.0155642 0.00583658 0.00972763 0.01167315 0.0077821 0.00972763
0.0077821 0.01945525 0.00583658 0.01750973 0.0155642 0.00389105
0.00194553 0.00389105 0.00972763 0.00972763 0.01167315 0.00972763
0.00389105 0.0077821 0.00389105 0.01361868 0.0077821 0.0077821
0.00583658 0.01750973 0.00583658 0.01167315 0.00583658 0.00194553
0.01945525 0.0077821 0.01361868 0.01167315]
```



As T increases, the empirical stationary distribution $\hat{\pi}$ will converge to the theoretical stationary distribution π . This is because, over a long period of time, the random walk will visit each node in proportion to its degree, as described by the stationary distribution. Therefore, the more steps we take in the random walk, the closer our empirical estimates will be to the theoretical values.

Double-click (or enter) to edit

▼ d)

Using your graph from part (a), build the random walk transition matrix $P = D^{-1}A$.

Use a linear algebra routine to compute:

1. The largest eigenvalue of P and its (right) eigenvector
2. The second-largest eigenvalue in magnitude and its eigenvector

```
1 # import eigs
2 from scipy.sparse.linalg import eigs
3
4 # Already defined P in part a.
5
6 # Using linear algebra to compute lambda_1 and its right eigenvector
7 vals, vecs = eigs(P.T, k=1, which='LR')
8 lambda1 = vals[0]
9 pi_vec = vecs[:, 0].real
10 # pi_vec /= np.sum(pi_vec) # Normalize to sum to 1
11 print("Lambda_1 from eigs:", lambda1)
12 print("Stationary distribution  $\pi$  from eigs:", pi_vec)
13
14 print("-----")
15
16 # compute the second largest eigenvalue
17 vals2, _ = eigs(P.T, k=2, which='LR')
18 lambda2 = np.sort(np.abs(vals2))[:-1][1]
```

```
19 print("Lambda_2 from eigs:", lambda2)
20
```

```

Lambda_1 from eigs: (1.0000000000000007+0j)
Stationary distribution  $\pi$  from eigs: [0.07060045 0.08825056 0.01765011 0.08825056 0.05295034 0.19415124
0.07060045 0.10590068 0.12355079 0.07060045 0.08825056 0.07060045
0.03530023 0.08825056 0.1412009 0.10590068 0.10590068 0.08825056
0.1412009 0.15885101 0.10590068 0.07060045 0.1412009 0.05295034
0.10590068 0.07060045 0.07060045 0.12355079 0.15885101 0.10590068
0.08825056 0.07060045 0.03530023 0.08825056 0.10590068 0.10590068
0.10590068 0.07060045 0.08825056 0.12355079 0.05295034 0.01765011
0.10590068 0.07060045 0.07060045 0.10590068 0.08825056 0.05295034
0.12355079 0.08825056 0.05295034 0.01765011 0.07060045 0.22945147
0.15885101 0.12355079 0.08825056 0.15885101 0.03530023 0.12355079
0.03530023 0.08825056 0.03530023 0.12355079 0.1412009 0.08825056
0.1412009 0.05295034 0.08825056 0.10590068 0.07060045 0.08825056
0.07060045 0.17650113 0.05295034 0.15885101 0.1412009 0.03530023
0.01765011 0.03530023 0.08825056 0.08825056 0.10590068 0.08825056
0.03530023 0.07060045 0.03530023 0.12355079 0.07060045 0.07060045
0.05295034 0.15885101 0.05295034 0.10590068 0.05295034 0.01765011
0.17650113 0.07060045 0.12355079 0.10590068]
-----
Lambda_2 from eigs: 0.7922074306978849

```

▼ e)

Normalize the eigenvectors so that their entries sum to 1 (to interpret them as distributions). Compare this normalized vector to the stationary distribution π from part (c) and comment on any similarities or differences.

```

1 # normalizing eigenvector
2 p1_vec_norm = pi_vec / np.sum(pi_vec) # Normalize to sum to 1
3 print("Stationary distribution  $\pi$  from eigs:", p1_vec_norm)
4 # compare it with the stationary distribution from part (a)
5 print("Empirical stationary distribution  $\pi^{\wedge}$ :", pi_hat)# compare lambda_1 with 1
6
7 # plot the difference between empirical and theoretical stationary distributions histogram
8 plt.figure(figsize=(10, 5))
9 plt.hist(p1_vec_norm - pi_hat, bins=20, color='lightcoral', edgecolor='black')
10 plt.title('Difference between Theoretical and Empirical Stationary Distributions', fontsize=16)
11 plt.xlabel('Difference', fontsize=14)
12 plt.ylabel('Frequency', fontsize=14)
13 plt.grid(True, alpha=0.3)
14 plt.tight_layout()
15 plt.show()

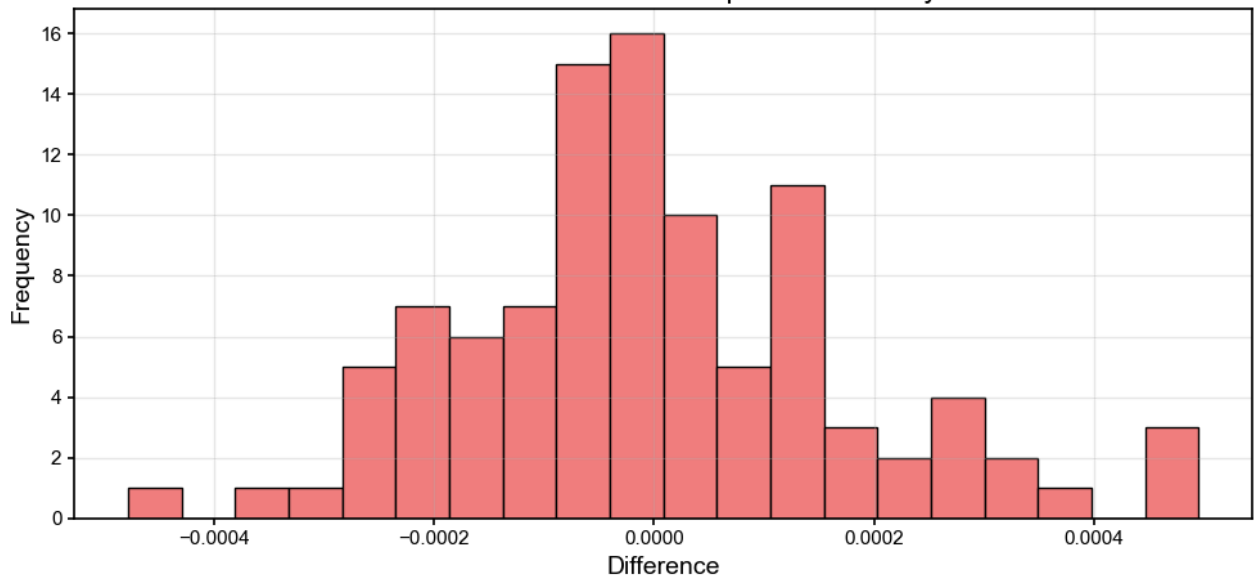
```

```

Stationary distribution  $\pi$  from eigs: [0.0077821 0.00972763 0.00194553 0.00972763 0.00583658 0.02140078
0.0077821 0.01167315 0.01361868 0.0077821 0.00972763 0.0077821
0.00389105 0.00972763 0.0155642 0.01167315 0.01167315 0.00972763
0.0155642 0.01750973 0.01167315 0.0077821 0.0155642 0.00583658
0.01167315 0.0077821 0.0077821 0.01361868 0.01750973 0.01167315
0.00972763 0.0077821 0.00389105 0.00972763 0.01167315 0.01167315
0.01167315 0.0077821 0.00972763 0.01361868 0.00583658 0.00194553
0.01167315 0.0077821 0.0077821 0.01167315 0.00972763 0.00583658
0.01361868 0.00972763 0.00583658 0.00194553 0.0077821 0.02529183
0.01750973 0.01361868 0.00972763 0.01750973 0.00389105 0.01361868
0.00389105 0.00972763 0.00389105 0.01361868 0.0155642 0.00972763
0.0155642 0.00583658 0.00972763 0.01167315 0.0077821 0.00972763
0.0077821 0.01945525 0.00583658 0.01750973 0.0155642 0.00389105
0.00194553 0.00389105 0.00972763 0.00972763 0.01167315 0.00972763
0.00389105 0.0077821 0.00389105 0.01361868 0.0077821 0.0077821
0.00583658 0.01750973 0.00583658 0.01167315 0.00583658 0.00194553
0.01945525 0.0077821 0.01361868 0.01167315]
Empirical stationary distribution  $\pi^*$ : [0.00752504 0.00995389 0.00210621 0.00961722 0.00601802 0.02164324
0.00780159 0.01132463 0.01386971 0.0077595 0.00976351 0.00783365
0.0039519 0.00982162 0.0156172 0.01202202 0.01127653 0.00974547
0.01575748 0.01741279 0.01174547 0.00772744 0.01554706 0.00576352
0.01169336 0.00778556 0.00783566 0.01365528 0.017519 0.01188174
0.00993184 0.00761121 0.0039018 0.00960519 0.01185769 0.01183164
0.01187974 0.00763125 0.00979357 0.0133627 0.0059579 0.00201002
0.01176952 0.00766331 0.00810018 0.01120839 0.00953706 0.00594788
0.01347292 0.00984768 0.00578155 0.00196392 0.00783566 0.02550095
0.0177675 0.0135631 0.00974146 0.01750898 0.00385971 0.01346691
0.00389378 0.0098116 0.00388777 0.01333865 0.01561921 0.00978756
0.01537071 0.00596993 0.00972944 0.01143284 0.00789778 0.00967934
0.00771742 0.01970136 0.00571742 0.01753503 0.01545087 0.00373747
0.00194188 0.00394989 0.00923245 0.00978756 0.01164126 0.00991782
0.00393987 0.00771141 0.00403005 0.01355308 0.00756712 0.00793385
0.00610219 0.01798794 0.00578356 0.01136871 0.00571542 0.00183166
0.01920036 0.00732864 0.01368334 0.01162723]

```

Difference between Theoretical and Empirical Stationary Distributions



The largest eigenvalue of the random-walk matrix P is always 1, and its corresponding eigenvector represents the stationary distribution of the random walk. When we normalize this eigenvector so that its entries sum to 1, it should closely match the theoretical stationary distribution π computed in part (c). This is because the stationary distribution describes the long-term behavior of the random walk, which is captured by the eigenvector associated with the largest eigenvalue.

✓ f)

Construct a path graph P_n on $n=30$ nodes, and a complete graph K_n on $n=30$ nodes.

For each graph, build the corresponding random walk transition matrix $P = D^{-1}A$ and compute the largest and second-largest eigenvalues. Then, compute the spectral gap $\gamma = \lambda_1 - \lambda_2$ for each graph.

```

1 n = 30
2 # Path graph
3 G_path = nx.path_graph(n)
4 A_path = nx.adjacency_matrix(G_path)
5 degrees_path = np.array([d for _, d in G_path.degree()], dtype=float)
6 degrees_path[degrees_path == 0.0] = 1.0
7 D_inv_path = sp.diags(1.0 / degrees_path)

```